

Lenguajes de Programación - Examen ejemplo

Marzo de 2026. Duración: 1 hora

Ejercicio 1

Utilizando la sintaxis de list comprehensions, escribe una sola línea de código que genere una lista llamada `mis_multiplos` que contenga el triple de cada número en el rango del 1 al 15 (ambos inclusive).

Solución

```
# Se utiliza range(1, 16) para incluir el número 15
mis_multiplos = [x * 3 for x in range(1, 16)]
```

Explicación:

- `range(1, 16)`: genera un iterador que devuelve de forma incremental los números del 1 al 15. En Python, el límite superior de `range` no se incluye, por lo que usamos 16 para llegar hasta el 15.
- `for x in range(1, 16)`: Es el bucle que itera sobre la secuencia generada por `range`.
- `x * 3`: Es la expresión que, dados los valores generados, los transforma en los valores de la nueva lista.

Ejercicio 2

Enunciado Escribe un programa que solicite al usuario un número entero para dividir el número 100 entre él. El programa debe:

- Gestionar la excepción `ValueError` en caso de que el usuario introduzca texto o caracteres no numéricos.

- Gestionar la excepción `ZeroDivisionError` en caso de que el usuario introduzca el número cero.
- Utilizar el bloque `else` para mostrar el resultado si la operación es exitosa.
- Utilizar el bloque `finally` para imprimir un mensaje indicando que el proceso ha terminado, independientemente de si hubo error o no.

Solución

```
try:
    # Solicitamos la entrada y la convertimos a entero
    divisor = int(input("Introduce un número entero para dividir 100: "))
    resultado = 100 / divisor
except ValueError:
    # Se ejecuta si la conversión int() falla
    print("¡Error! Debes introducir un número válido.")
except ZeroDivisionError:
    # Se ejecuta si el divisor es 0
    print("¡Error! No puedes dividir entre cero.")
else:
    # Solo se ejecuta si no ocurrió ninguna de las excepciones anteriores
    print(f"Resultado exitoso: {resultado}")
finally:
    # Se ejecuta siempre al final
    print("Fin del proceso de cálculo.")
```

Puntos clave de la solución:

- `int(input(...))`: Se utiliza para recibir datos del usuario y transformarlos al tipo de dato básico entero.
- Orden de los bloques: Es fundamental que los bloques `except` específicos vayan antes que un posible bloque genérico y que `else` y `finally` cierren la estructura.
- `f"Resultado: {resultado}"`: Se utiliza una f-string para formatear la salida de datos de forma clara.

Ejercicio 3

Escribe una función generadora llamada `pares_hasta(n)` que reciba un número entero `n` y entregue, de uno en uno, todos los números pares que se encuentren en el rango desde 0 hasta `n` (inclusive). Después, utiliza un bucle para consumir los valores del generador e imprimirlos.

Solución

```
def pares_hasta(n):
    for i in range(n + 1):
        if i % 2 == 0:
            # yield devuelve el valor y pausa la ejecución de la función
            yield i

# Se obtiene el objeto generador
generador_de_pares = pares_hasta(10)

# Se itera sobre el objeto generador para obtener los valores
for numero in generador_de_pares:
    print(numero)
```

Puntos clave:

- Ahorro de memoria: A diferencia de crear una lista con todos los números pares, esta función solo mantiene un elemento en memoria cada vez.
- Objeto generador: Al llamar a `pares_hasta(10)`, no se ejecuta el código inmediatamente, sino que se devuelve un objeto de la clase 'generator' sobre el cual se puede iterar.
- Control de flujo: El proceso termina automáticamente cuando se agotan los elementos del rango. Si intentáramos pedir un valor adicional manualmente después de terminar, el intérprete lanzaría la excepción `StopIteration`.

Ejercicio 4

La empresa Pacotilla, S.L., que se dedica a la entrega de paquetes (mensajería), quiere informatizar su negocio. La empresa gestiona paquetes, que tienen una ciudad de entrega, una ciudad remitente, un peso y un coste. La empresa dispone de camiones que reparten paquetes a las diferentes direcciones. Cada camión tiene una matrícula, una lista de paquetes que entregar y una capacidad de paquetes.

Se pide:

- Modelar las clases necesarias
- Implementar el método `add_paquetes` en la clase `Camion` que verifique que el peso total de los paquetes no supera la capacidad del camión. En caso contrario se elevará la excepción `ValueError`. Se valorará no usar bucles.
- Implementar un método generador `ciudades` en `Camion` que devuelva cada una de las ciudades destino de cada paquete, de una en una.

Solución

```
class Paquete:
    def __init__(self, dir, remite, peso):
        self._dir = dir
        self._remite = remite
        self._peso = peso

class Camion:
    def __init__(self, matricula, capacidad):
        self._matricula = matricula
        self._capacidad = capacidad

    def add_paquetes(self, paquetes):
        pesos = map(lambda p: p._peso, paquetes)
        if sum(pesos) > self._capacidad:
            raise ValueError("El peso excede la capacidad del camión")
        self._paquetes = paquetes

    def ciudades(self):
        for p in self._paquetes:
            yield p._dir

p1 = Paquete("Madrid", "Pamplona", 40, 35.6)
p2 = Paquete("Ciudad Real", "Albacete", 80, 75.6)

c = Camion("0000mna", 130)

c.add_paquetes([p1, p2])

for ciudad in c.ciudades():
    print(ciudad)
```

Puntos clave:

- Hay que especificar siempre **self** como primer parámetro de los métodos
- Hay que inicializar los atributos que corresponda en el constructor (método `__init__()`), y éstos deben comenzar por `_` (guión bajo) y estar cualificados con **self**.
- En el método `add_paquetes` usamos la función predefinida `map` para construir una lista con los pesos de cada paquete a partir de la lista de paquetes
- Usamos `sum` para sumar los valores de la lista y compararlos con la capacidad del camión